

Report: Regulatory Activity of DNA

1. Introduction

The goal of this project was to predict regulatory activity from DNA sequences. The dataset contained DNA sequences and two target variables. The first target, `is_active`, was a binary classification label indicating whether a sequence was regulatory active. The second target, `rna_dna_ratio`, was a continuous value describing the regulatory strength of the sequence.

I approached this as a multitask learning problem and built a hybrid model combining a convolutional neural network with LightGBM models. The CNN was used to learn sequence representations directly from one-hot encoded DNA, while LightGBM was used on top of CNN embeddings and additional handcrafted DNA features.

2. Data preprocessing

The DNA sequences were represented using one-hot encoding. Each nucleotide was encoded as one of four channels corresponding to A, C, G, and T. The sequence length was fixed to 230 nucleotides, which matched the length of the sequences in the dataset. Missing values in the sequence and target columns were removed before training.

I kept the full sequences without adapter trimming. I tested the preprocessing setup and the model performed better when the full sequence was used, so I set adapter trimming lengths to zero. The regression target, `rna_dna_ratio`, was standardized during CNN training to make optimization more stable. During validation and prediction, the regression outputs were transformed back to the original scale.

3. Model architecture

The main neural network was inspired by DeepSTARR-like architectures for regulatory sequence modeling. It was a 1D convolutional neural network that processed one-hot encoded DNA sequences. The model consisted of an initial convolutional stem followed by several convolutional blocks with batch normalization, GELU activation, dropout, max pooling, and residual skip connections.

After the convolutional feature extractor, I used global average pooling and a shared fully connected layer. From this shared representation, the model had two separate output heads:

- one classification head for predicting `is_active`,
- one regression head for predicting `rna_dna_ratio`.

The classification head produced a single logit and was trained using binary cross-entropy with logits. Because the classes were imbalanced, I used a positive class weight in the loss function. The regression head produced a continuous value and was trained using mean squared error loss.

The final CNN loss was a weighted sum of the classification loss and regression loss. This allowed the model to learn both tasks at the same time.

4. Data augmentation

I used reverse complement augmentation during CNN training. For each training sequence, the reverse complement was randomly used with probability 0.5. This augmentation is biologically motivated because DNA motifs can often be informative on either strand.

I also used reverse complement test-time augmentation. During validation and final prediction, I predicted each sequence twice: once in its original orientation and once as its reverse complement. The final CNN prediction was the average of both predictions. This made the model more robust to strand orientation.

5. Additional LightGBM models

After training the CNN, I used it as a feature extractor. I extracted the learned representation from the shared CNN feature layer for each sequence. These CNN embeddings were then used as input features for LightGBM models.

I also added handcrafted DNA features, including nucleotide frequencies, GC content, and normalized 2-mer, 3-mer, and 4-mer frequencies. These k-mer features describe short sequence patterns and can help LightGBM capture motif-like information that may complement the CNN embeddings.

I trained two LightGBM models:

- a LightGBM classifier for `is_active`,
- a LightGBM regressor for `rna_dna_ratio`.

For both models, I used features from the original sequence and the reverse complement sequence. During validation, I averaged predictions from both orientations.

6. Experiments and model selection

During the project, I tested several approaches. First, I trained a standalone CNN model using only one-hot encoded DNA sequences. This gave reasonable results, but performance improved after adding reverse complement augmentation and test-time averaging.

I also tried using pretrained DNA language models from Hugging Face, such as DNABERT-like models. However, I had technical problems with downloading and

loading these models correctly in my environment, so I decided not to use them in the final solution.

I also tested LightGBM as a standalone model using handcrafted DNA features such as nucleotide composition, GC content, and k-mer frequencies. This approach was useful, but the best results were obtained when LightGBM was combined with CNN embeddings.

The final selected solution was a hybrid CNN + LightGBM model. The CNN learned sequence-level representations, while LightGBM used these learned features together with handcrafted DNA features. This approach gave the best validation performance among the tested variants.

7. Final prediction strategy

For classification, I combined CNN test-time augmented probabilities with LightGBM classifier probabilities using a weighted average. The final classification threshold and ensemble weight were selected on the validation set.

For regression, I combined the CNN regression prediction with the LightGBM regression prediction using a weighted average. The regression ensemble weight was also selected based on validation performance.

The final model therefore used:

- CNN sequence embeddings,
- reverse complement augmentation,
- reverse complement test-time averaging,
- handcrafted DNA features,
- LightGBM classifier and regressor,
- weighted ensembling of CNN and LightGBM predictions.

8. Validation results

On the validation set, the final hybrid model achieved approximately the following results:

Plain CNN with a default threshold of 0.5 achieved accuracy around 0.71–0.73 and AUC around 0.80–0.81.

Applying threshold tuning improved accuracy to approximately 0.75, although at the cost of a lower F1 score.

CNN with reverse complement test-time augmentation (TTA) further improved performance, reaching accuracy around 0.73–0.75 (or up to ~0.76 after threshold tuning) and AUC around 0.81–0.82.

The LightGBM classifier trained on CNN embeddings achieved comparable performance, with accuracy around 0.75–0.76 and AUC around 0.80–0.81.

The final CNN + LightGBM ensemble provided the best overall results, reaching accuracy around 0.76–0.77, F1 score around 0.60–0.61, AUC around 0.81–0.82, and regression MSE around 0.37–0.39.

The validation results showed that the hybrid approach improved over the plain CNN and gave more robust predictions.

9. Limitations

The limitation was that I could not successfully use pretrained DNA language models because of technical issues with loading them. Such models could potentially improve performance if they were properly integrated.

10. Conclusion

The final solution is a hybrid model combining a DeepSTARR-inspired CNN with LightGBM models. The CNN learns regulatory patterns directly from DNA sequences, while LightGBM uses CNN embeddings and handcrafted k-mer features. Reverse complement augmentation and test-time averaging improve biological robustness.

I selected this approach because it performed better than the standalone CNN, standalone LightGBM, and the pretrained-model attempts I tested. The final model provides both classification predictions for regulatory activity and regression predictions for regulatory strength.

11. Evaluation script usage

For evaluation, I provide an `evaluation_script.py` file. The script should be run with two arguments: the path to the folder containing the saved configuration and model files, and the path to the test dataset.

The model folder should contain:

- - `final_config.json`
- - `best_model_deep.pth`
- - `lgb_classifier.pkl`
- - `lgb_regressor.pkl`

The script can be run as:

```
python evaluation_script.py <path_to_folder_with_config_and_models>  
<path_to_test_data>
```

For example, if all saved files are in the current directory, it can be run as:

```
python evaluation_script.py . test_data.tsv
```

